

PROJECT SCOPING DOCUMENT

ClaimSense

Know what your insurance is actually worth, reconciling hospital bills against plan rules, one line item at a time.

Django course group project · v0.3, revised to match the Part 4 data model build · prepared for team review before P1-A1

WHY WE'RE BUILDING THIS

The problem

Nobody finds out whether their health insurance is actually good until a bill lands in their mailbox and they're staring at a number they don't understand. Deductibles, coinsurance percentages, in-network vs. out-of-network status, and out-of-pocket maximums all interact quietly, and the hospital bill itself just shows a total. Most people have no easy way to check the math or to know, across a whole year, whether their plan is pulling its weight.

ClaimSense closes that gap. A user enters what a hospital charged them, line item by line item, along with the terms of their insurance plan. Each line item records an estimated covered amount and an estimated patient responsibility, so the app can show exactly what the plan is expected to pick up versus what the patient is on the hook for. Over multiple bills, it builds a simple track record: how much this plan has paid out relative to what was charged, so "is my insurance good?" gets an actual answer instead of a shrug.

WHO IT'S FOR

Users & use case

The primary user is anyone who pays for their own healthcare and wants clarity before or after a visit, a student on a campus plan, a young professional comparing employer plan options, or anyone tired of guessing whether a bill looks right. The core loop is short: log a bill, see the breakdown, watch the running coverage picture update. Nothing about the workflow assumes insurance expertise; the app is meant to translate plan language into a plain answer.

BOUNDARIES

Scope

We scoped this deliberately tight for a three-week build. Two lists below: what ships, and what we're explicitly leaving out so nobody quietly tries to build it anyway.

In scope

- Manual entry of bill line items, description, procedure reference, charged amount, and a line number identifying its place on the bill.
- One active insurance plan per patient, defined by plan type (PPO/HMO/EPO/Other), deductible, coinsurance percentage, and out-of-pocket maximum.
- Per-line-item coverage fields (estimated covered amount, estimated patient responsibility), entered and validated in Django Admin for this milestone; automatic calculation from plan rules is the next build step.
- A running coverage history and a plan usefulness score, both viewable as a chart.
- Search and filtering across a user's bills, plus a JSON API for their claims data.

Out of scope, for this course

- OCR or photo upload of a real bill, real scope risk for the time we have; structured manual entry gets us the same data model.

- Comparing multiple insurance plans side by side; the schema allows it later, but v1 assumes one plan per patient.
- Any real integration with an actual insurer or claims clearinghouse; all coverage math is computed or entered locally.
- Authentication beyond what the course requires, no password reset flows, no third-party login.

FUNCTIONALITY

Core features

Feature	What it does
Bill entry	Add a bill with a reference number, patient, provider, insurance plan, and bill date. Each bill carries one or more line items (line number, description, procedure reference, charged amount).
Coverage fields	Each line item stores an estimated covered amount and an estimated patient responsibility alongside the charged amount, so the split between what the plan picked up and what the patient owes is recorded per line, not just per bill.
Dashboard	One page showing total charged vs. total estimated covered across all bills, a coverage-over-time chart, and the plan's usefulness score.
Search & filter	Filter bills by provider or date range; view aggregate totals per provider and per month using Django's ORM aggregation.
Claims API	A JSON endpoint returning a user's bills and their coverage outcomes, filterable by date and provider, the same data the dashboard renders, exposed for reuse.

DATA MODEL

Entities & relationships

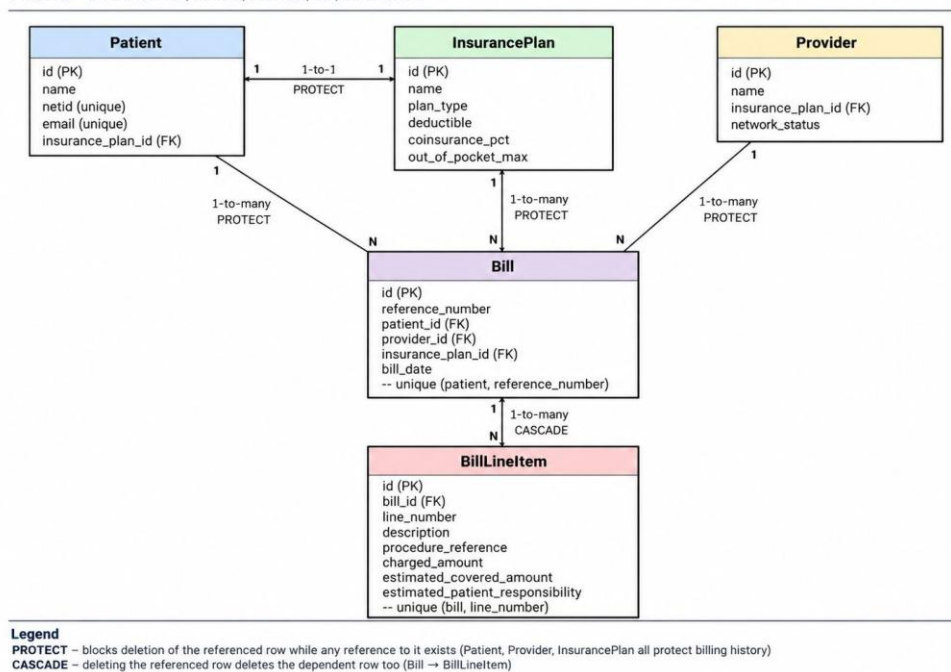
Five models carry the whole system, matching what's actually running in Django Admin as of the Part 4 build. A Patient is linked to exactly one InsurancePlan (a one-to-one relationship, not just a business rule), a Provider's network status is recorded per plan rather than globally, and a Bill ties together a Patient, Provider, and InsurancePlan, with its BillLineItems carrying the actual charge and coverage figures.

Model	Purpose & key fields
InsurancePlan	name, plan_type (PPO / HMO / EPO / Other), deductible, coinsurance_pct, out_of_pocket_max, the rules a bill gets checked against.
Patient	name, netid (unique), email (unique), insurance_plan (one-to-one FK, PROTECT), one row per user of the app, linked to exactly one active plan.
Provider	name, insurance_plan (FK, PROTECT), network_status (In Network / Out of Network), a provider's network status is recorded per plan, so the same physical provider can carry a different status under a different plan. Unique on (name, insurance_plan).
Bill	reference_number, patient FK (PROTECT), provider FK (PROTECT), insurance_plan FK (PROTECT), bill_date, the top-level record for one invoice. Unique on (patient, reference_number), so the same reference can't be logged twice for one patient.
BillLineItem	bill FK (CASCADE), line_number, description, procedure_reference, charged_amount, estimated_covered_amount, estimated_patient_responsibility, one row per charge. Unique on (bill, line_number). Deletion cascades from Bill, since a line item has no meaning without its parent bill.

ER diagram

ClaimSense – Entity Relationship Diagram

5 models – InsurancePlan, Patient, Provider, Bill, BillLineItem



PROTECT is used for Patient, Provider, and InsurancePlan wherever a Bill references them, so a historical billing record never loses the entity it depended on. CASCADE runs from Bill to BillLineItem, since a line item has no meaning without its parent bill.

TIMELINE

Milestones

The build follows the course's own assignment cadence, each one is a real slice of the finished app, not a throwaway exercise.

Due	Assignment	What we ship
Sept 7	P1-A1	Team contract signed; project idea + wireframe PDFs submitted; Patient, InsurancePlan, Provider, Bill, and BillLineItem models running with real test data in Django Admin.
Sept 7	P1-PRF1	Individual CATME peer review of teammates' contribution on P1-A1.
Sept 14	P1-A2	GitHub repo live with .env hygiene, .gitignore, and a merged feature branch; bill list/detail pages working in all four required view styles; shared base template with navigation.
Sept 21	P1-A3	Search and filters on bills; totals and grouped coverage aggregates; a Matplotlib coverage-ratio chart served as an image; a working bill-submission form with CSRF; a filterable JSON claims API.

STILL DECIDING

Open questions for the team

- Should bill entry stay manual, or support photo/PDF upload? Lean: manual entry only for this course; OCR is a real feature but a real time sink.
- Do we support more than one insurance plan per patient? Lean: no, the Patient model's one-to-one link to InsurancePlan is deliberate for v1.
- How exactly is the "usefulness score" defined? Lean: covered ÷ charged, aggregated monthly.
- Should Provider stay scoped to a single plan, or should network status move to a join table so the same hospital can be shared across patients on different plans? Lean: keep the current per-plan Provider model for this course; flag it as a known simplification rather than redesign it under deadline.

IMPLEMENTATION

Tech stack

Django 5.2 on SQLite for the assignment's lifetime, Django Admin for data entry and validation, Matplotlib for the coverage chart (rendered server-side and served as PNG via BytesIO), python-dotenv for secrets, and Django's JsonResponse for the API layer. Nothing here is exotic on purpose, the goal is a clean, working data-driven app, not a technology showcase.

WATCH FOR

Risks & assumptions

- Coverage math has real edge cases (deductible partially met mid-bill, out-of-pocket max reached mid-year); we should agree on a simplified but consistent rule set early rather than debate it line item by line item.
- The usefulness score is only as good as the test data we seed; plan to enter a realistic spread of bills (some fully covered, some capped, some out-of-network) so the chart and aggregation demos actually show something meaningful.
- Provider being scoped per insurance plan means the same physical hospital needs a separate Provider row for each plan it appears under; worth watching as bill volume grows.
- Keep the plan and bill entry forms simple; the assignment rewards a clean, working request → view → template → response flow more than a feature-rich UI.